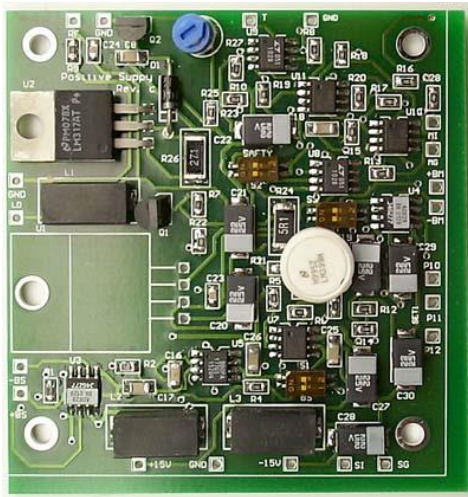


Software Modeling using UML

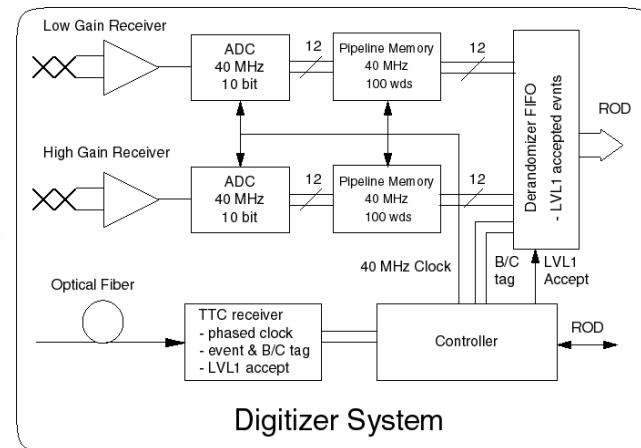
Engineering Models

► Engineering model:

A reduced representation of some system that highlights the properties of interest from a given viewpoint



Modeled system



Functional Model

- ◆ We don't see everything at once
- ◆ We use a representation (notation) that is easily understood for the purpose on hand

Models

- ▶ How do we model?
- ▶ Modeling Maturity Level
 - ▶ Level 0: No specification
 - ▶ Level 1: Textual
 - ▶ Level 2: Text with Diagrams
 - ▶ Level 3: Models with Text
 - ▶ Level 4: Precise Models

Unified Modeling Language (UML)

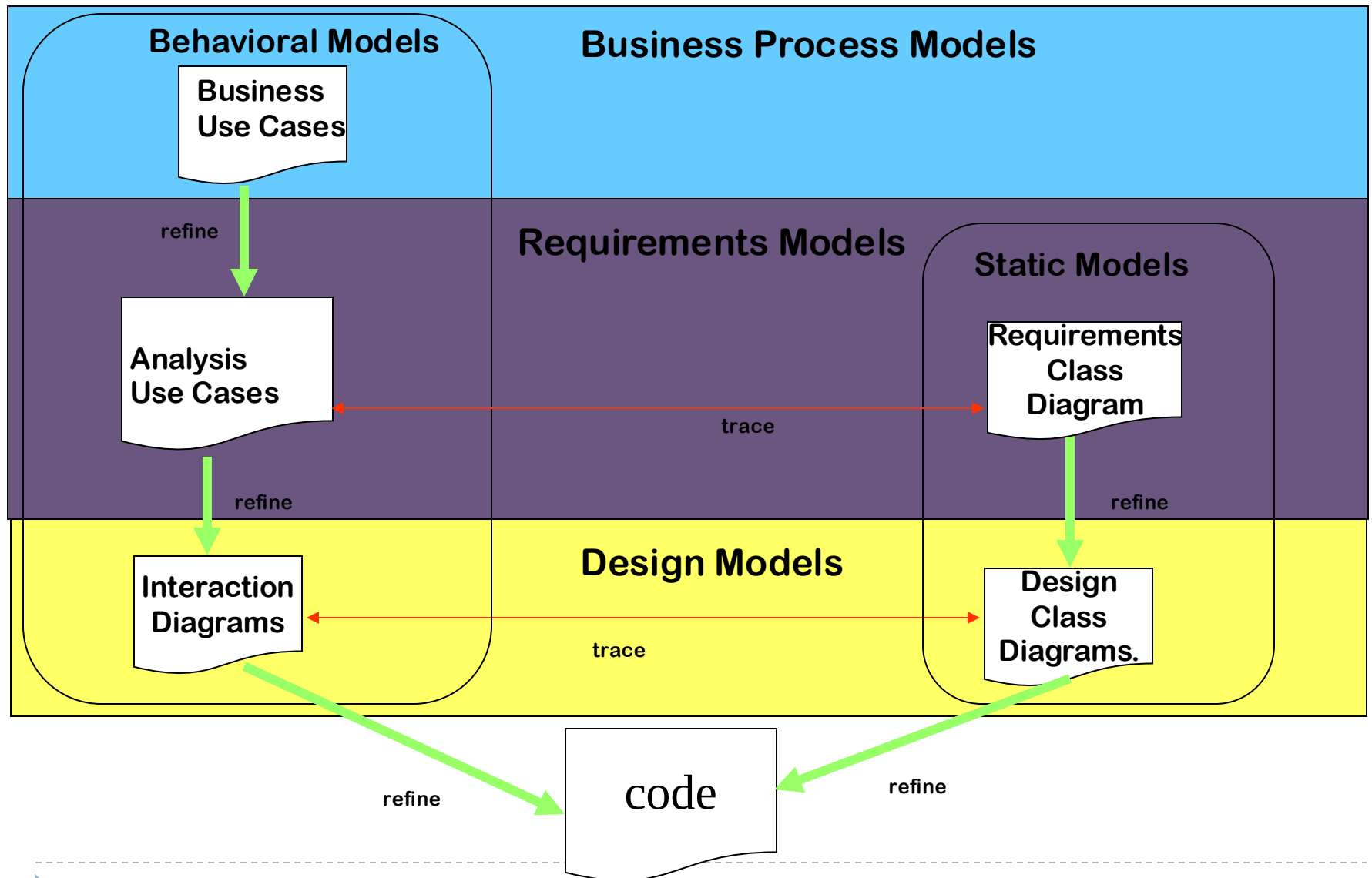
- ▶ Notation for object-oriented modeling
- ▶ Standardized by Object Management Group (OMG)
- ▶ Consists of 12+ different diagrams
 - ▶ Use case diagram
 - ▶ Class diagram
 - ▶ Statechart diagram
 - ▶ Sequence diagram
 - ▶ Communication diagram
 - ▶ Component diagram
 - ▶ ...

What the UML is not

- ▶ Not an OO method or process
- ▶ Not a visual programming language
- ▶ Not a tool specification



An “UltraLite” Process and its artifacts

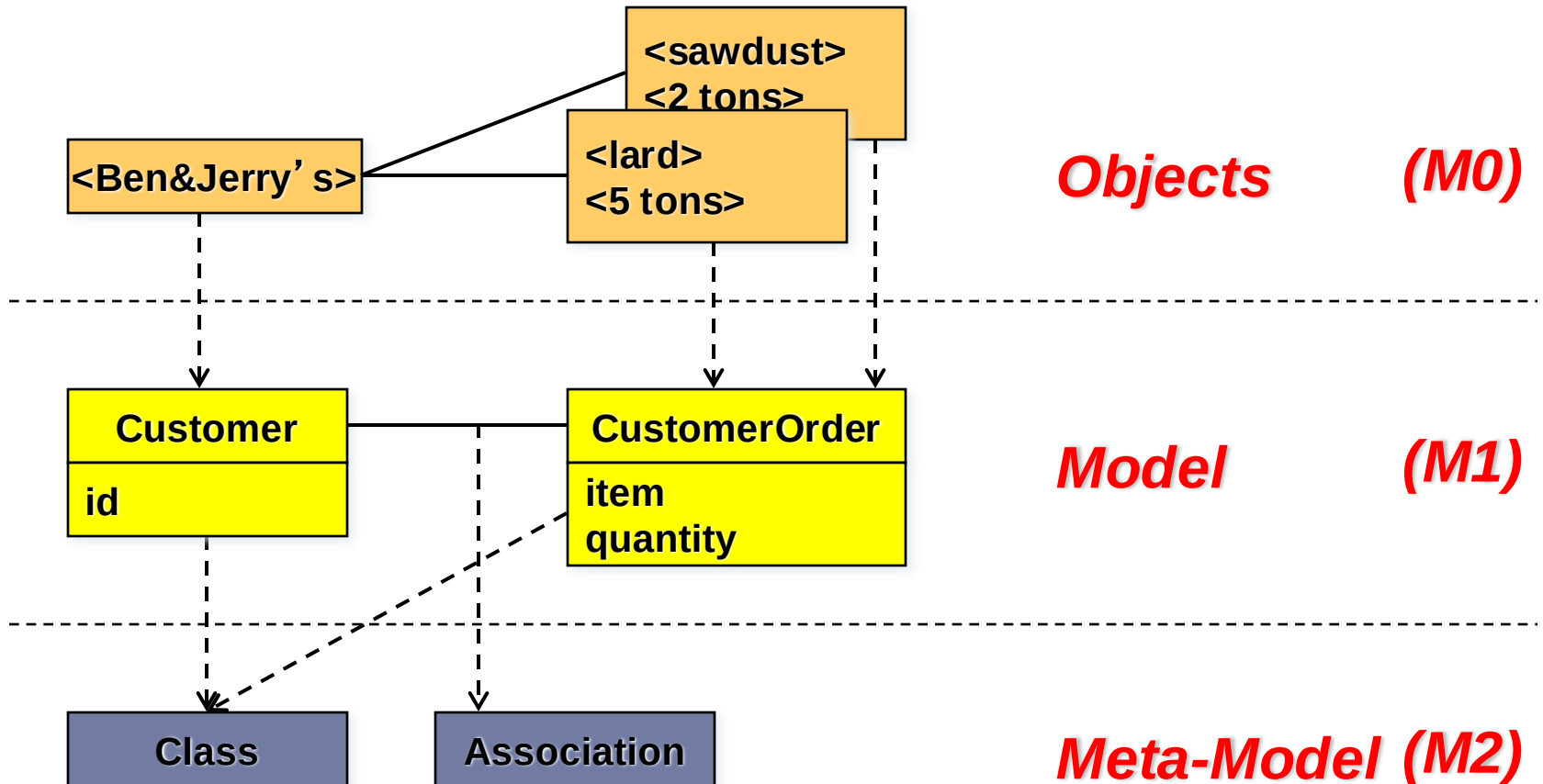


Static vs. Dynamic Models

- ▶ Static model
 - ▶ Describes static structure of a system
 - ▶ Consists of a set of objects (classes) and their relationships
 - ▶ Represented as class diagrams
- ▶ Dynamic model
 - ▶ Describes dynamic behavior of a system, such as state transitions and interactions (message sends)
 - ▶ Represented as statechart diagram, sequence diagrams, and collaboration diagrams

Models and Meta-Models

- ▶ Meta-models are simply Models of Models



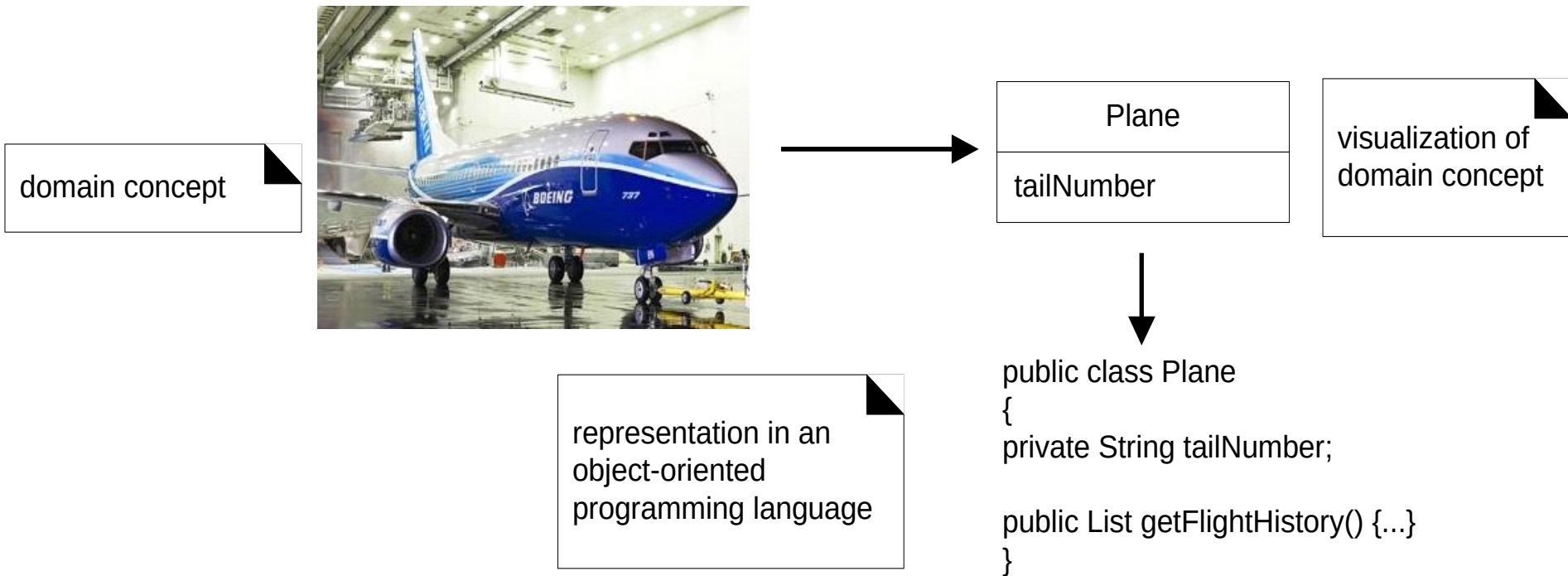
Domain concept

vs.

design representation of domain concept

vs.

code representation of domain concept

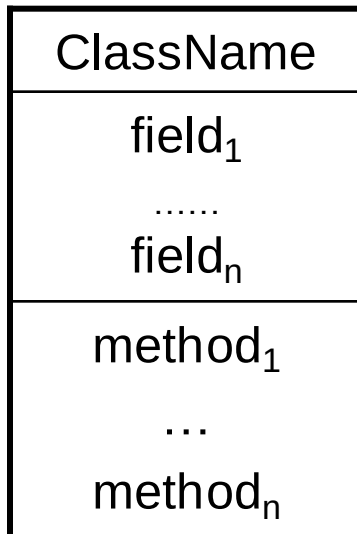


UML Class Diagram

- ▶ Most common diagram in OO modeling
- ▶ Describes the static structure of a system
- ▶ Consist of:
 - ▶ Nodes representing classes
 - ▶ Links representing of relationships among classes
 - ▶ Inheritance
 - ▶ Association, including aggregation and composition
 - ▶ Dependency

Notation for Classes

- ▶ The UML notation for classes is a rectangular box with as many as three compartments.



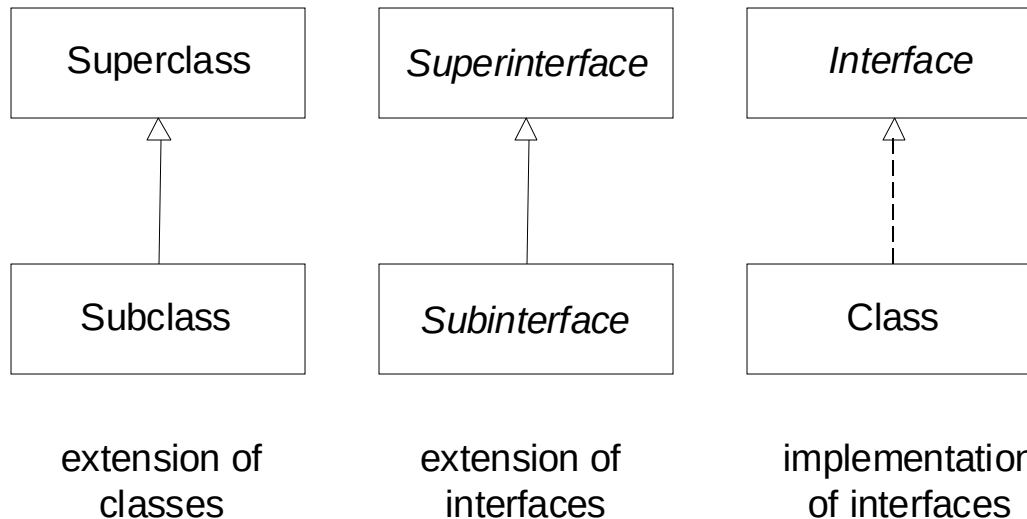
The top compartment show the class name.

The middle compartment contains the declarations of the fields, or *attributes*, of the class.

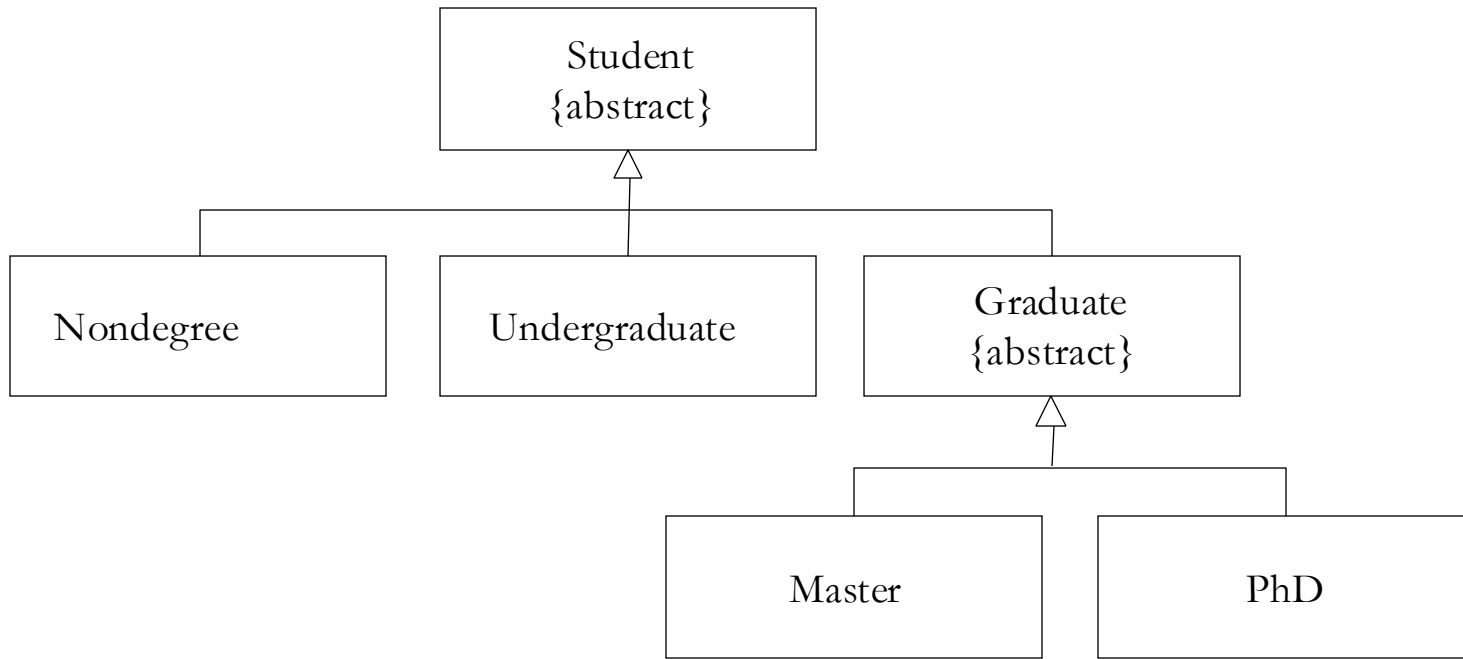
The bottom compartment contains the declarations of the methods of the class.

Inheritance in UML

- ▶ An extension relation is called *specialization* and *generalization*.
- ▶ An implementation relation is called *realization*.

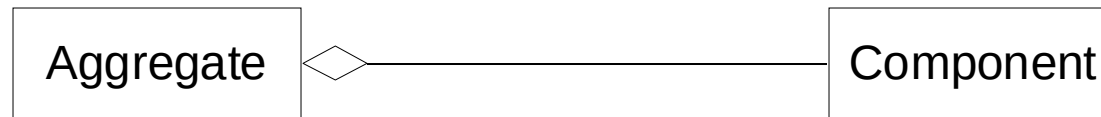


Inheritance - Example



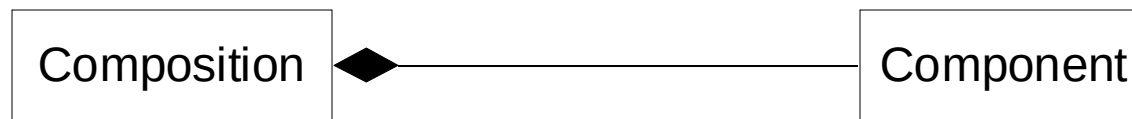
Aggregation

- ▶ Special form of association representing *has-a* or *part-whole* relationship.
- ▶ Distinguishes the whole (aggregate class) from its parts (component class).
- ▶ No relationship in the lifetime of the aggregate and the components (can exist separately).

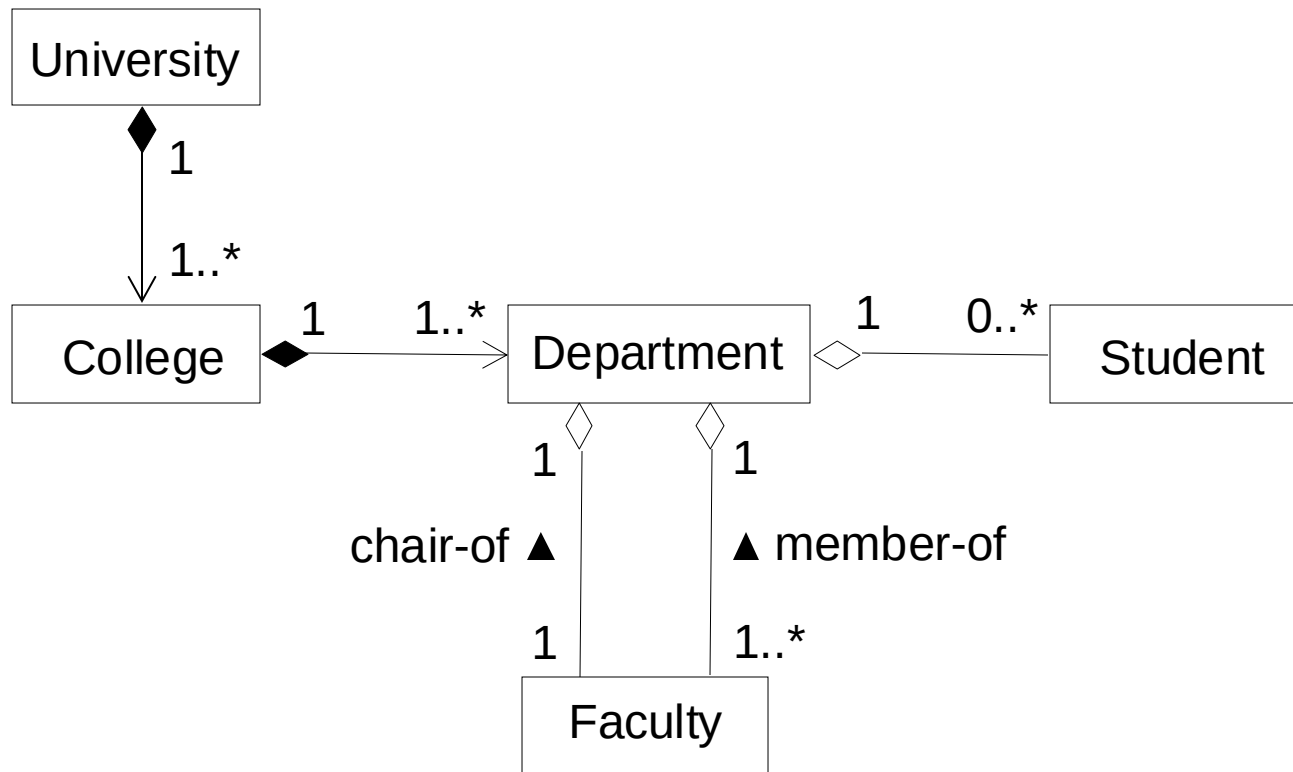


Composition

- ▶ Stronger form of aggregation
- ▶ Implies exclusive ownership of the component class by the aggregate class
- ▶ The lifetime of the components is entirely included in the lifetime of the aggregate (a component can not exist without its aggregate).

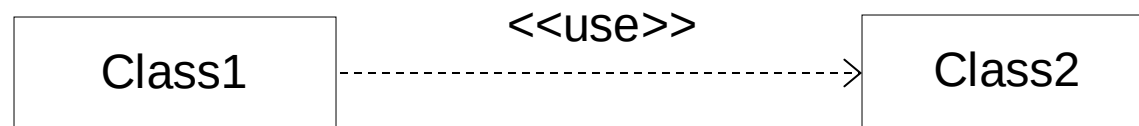


Aggregation - Example

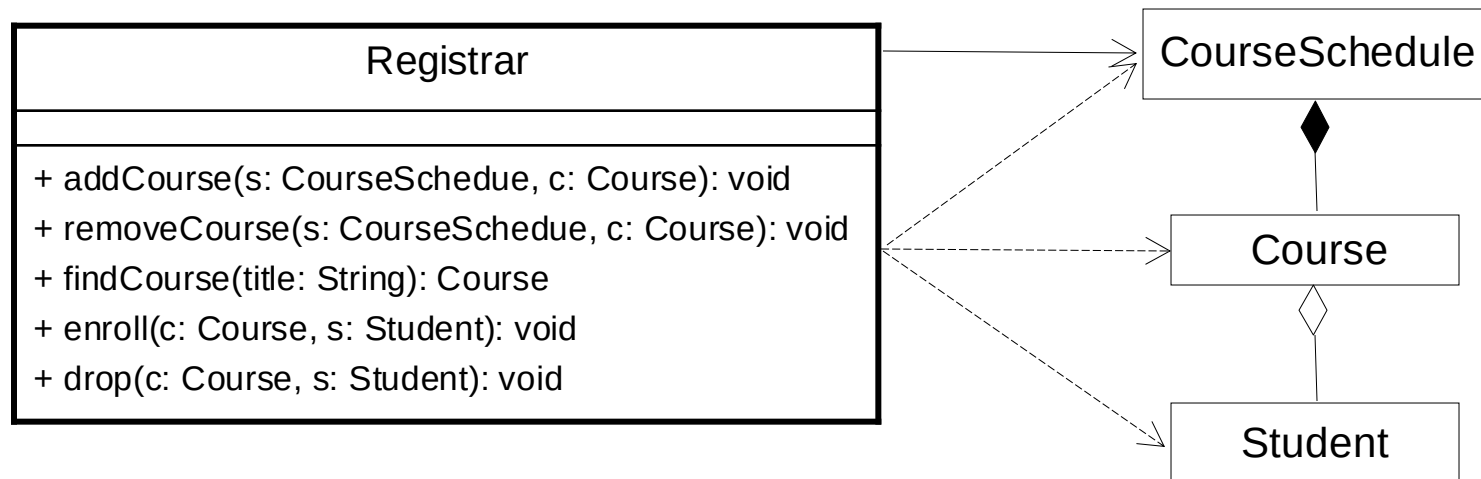


Dependency

- ▶ Relationship between the entities such that the proper operation of one entity depends on the presence of the other entity, and changes in one entity would affect the other entity.
- ▶ The common form of dependency is the *use* relation among classes.



Dependency - Example



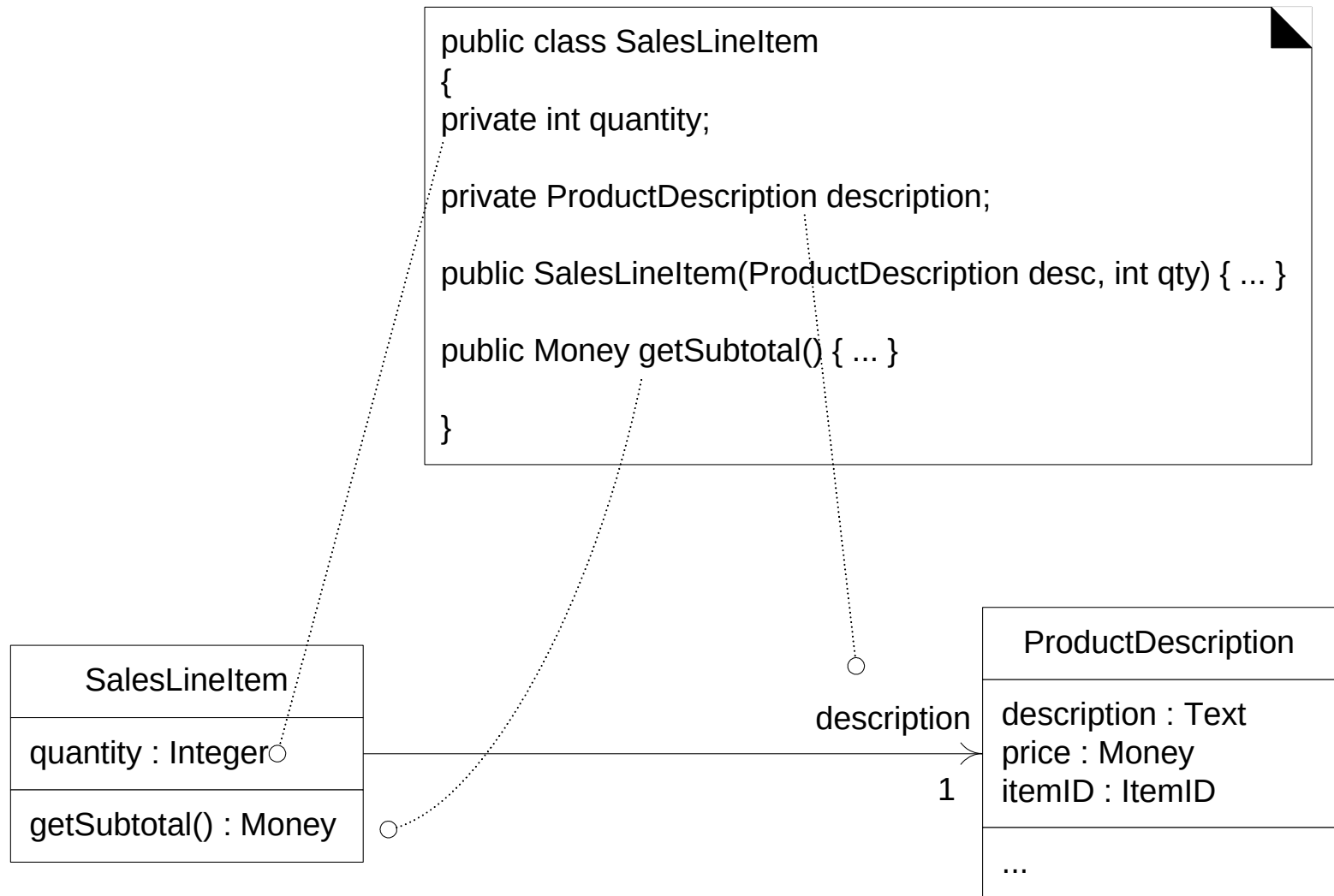
Dependencies are most often omitted from the diagram unless they convey some significant information.

Model to Model transformation

- ▶ Let's do a simple UML class diagram for the following piece of code written in Java

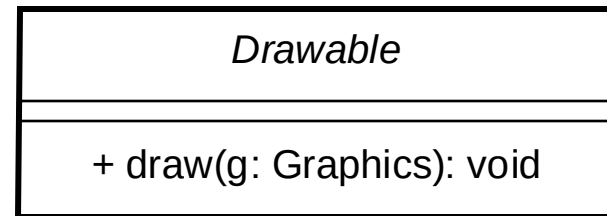
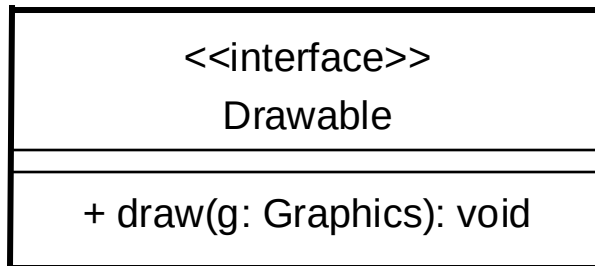
```
class Person {  
    private String name;  
    private Date birthday;  
    public String getName() {  
        // ...  
    }  
    public Date getBirthday() {  
        // ...  
    }  
}
```

Mapping UML class diagram to code



UML Notation for Interfaces

```
interface Drawable {  
    void draw(Graphics g);  
}
```



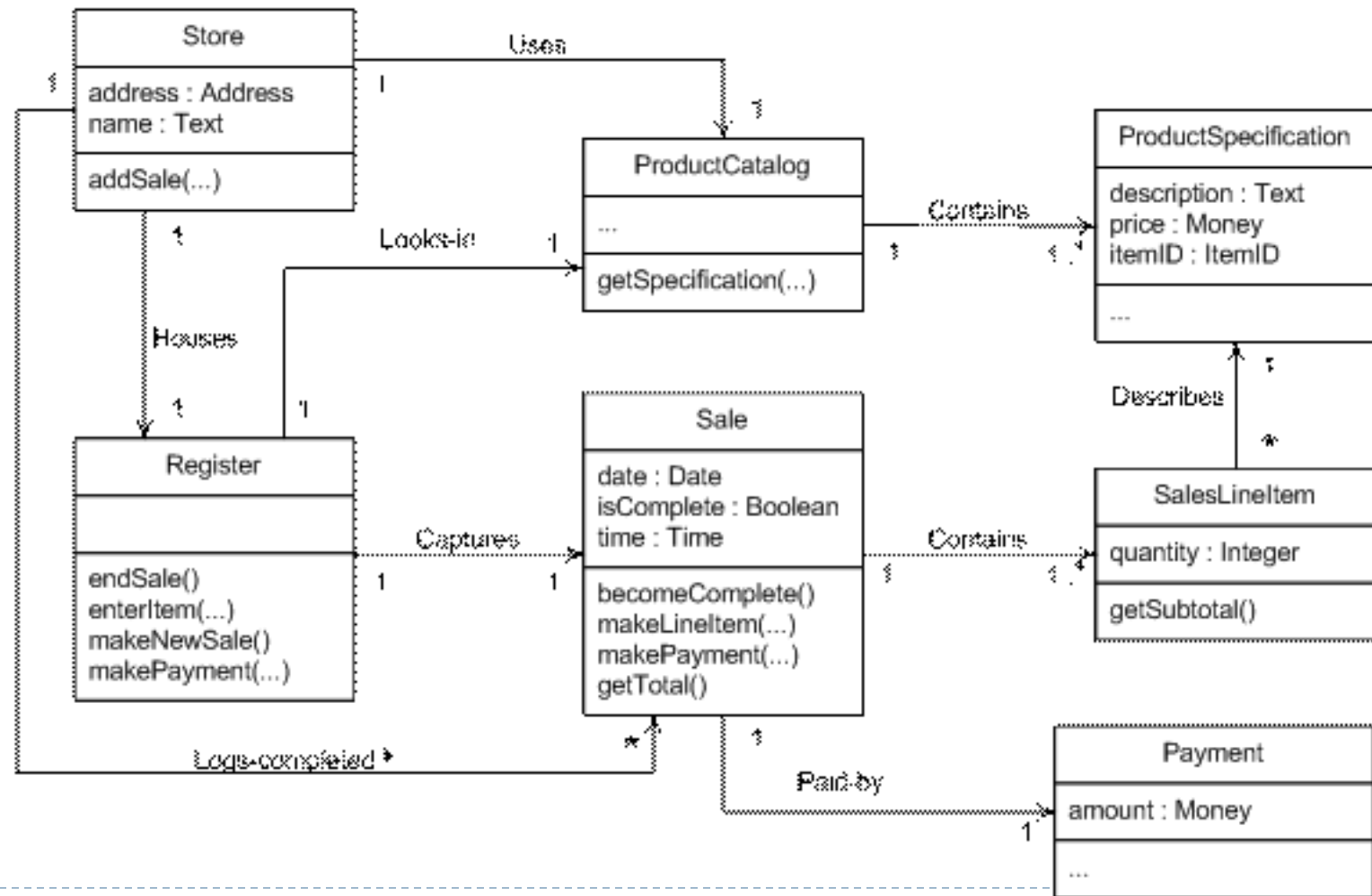
Suggested activities

- ▶ Identify a first set of candidate **classes**
- ▶ Add **associations** and **attributes**
- ▶ Find **generalizations**
- ▶ List the main **responsibilities** of each class
- ▶ Decide on specific **operations**
- ▶ **Iterate** over the entire process until the model is satisfactory
 - ▶ Add or delete classes, associations, attributes, generalizations, responsibilities or operations
 - ▶ Identify interfaces
 - ▶ Apply design patterns

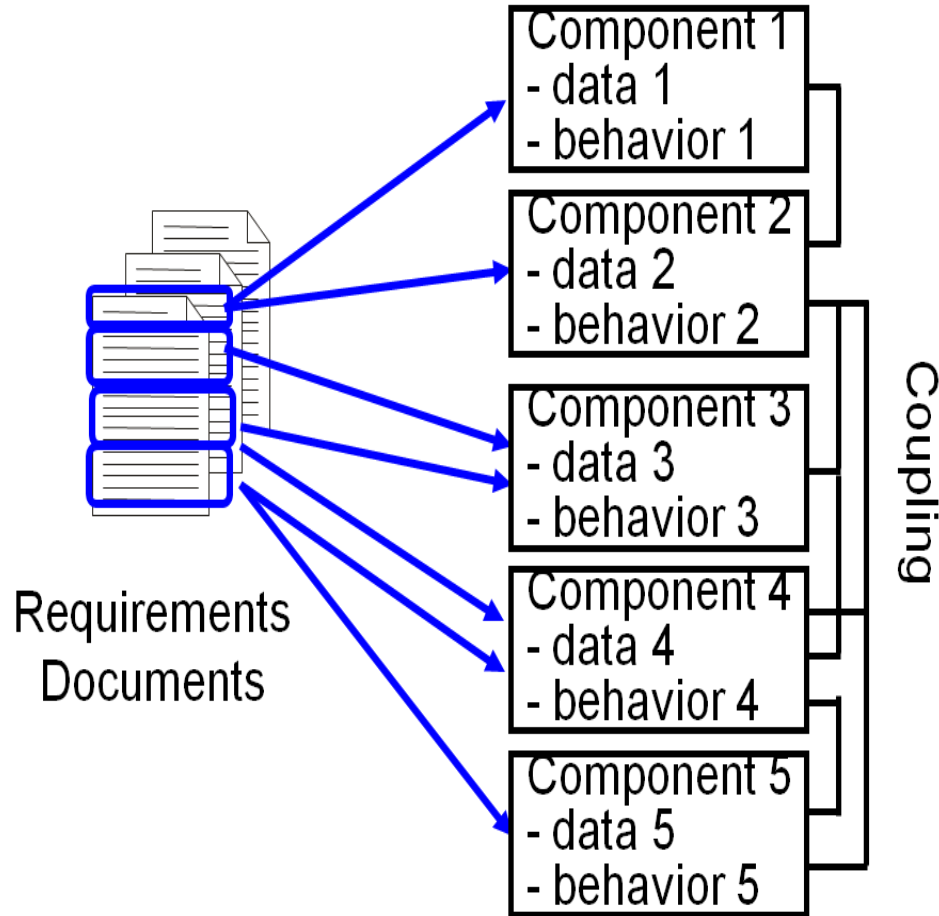
Don't be too disorganized. Don't be too rigid either!



Remember – POS Class Diagram ???



Mapping Requirements to Design Components



- ▶ Design must satisfy requirements
 - ▶ Everything (data and behavior) in the requirements must be mapped to the design components
 - ▶ Decide what functionality goes into which component
- ▶ As you do the mapping, assess functional cohesion and coupling
 - ▶ Strive for **loose** coupling and **high** cohesion

Group Exercise: E-book Store (two per team)

Develop an OO model for an e-bookstore. The core requirements of the e-bookstore are to allow its customers to browse and order books, music CDs, and computer software through the Internet. The main functionalities of the system are to provide information about the titles it carries to help customers make purchasing decisions; handle customer registration, order processing, and shipping; and support management of the system, such as adding, deleting, and updating titles and customer information.

1. Identify classes. Classes can represent physical objects, people, organizations places, events, or concepts. Class names should be noun phrases.
2. Identify relevant fields and methods of the classes. Actions are modeled as the methods of classes. Method names should be verb phrases.
3. Identify any inheritance relationships among the classes and draw the class diagram representing inheritance relationships.
4. Identify any association relationships among the classes and draw the class diagram representing association relationships.
5. Identify any aggregation and composition relationships among the classes and draw the class diagram representing dependency relationships.

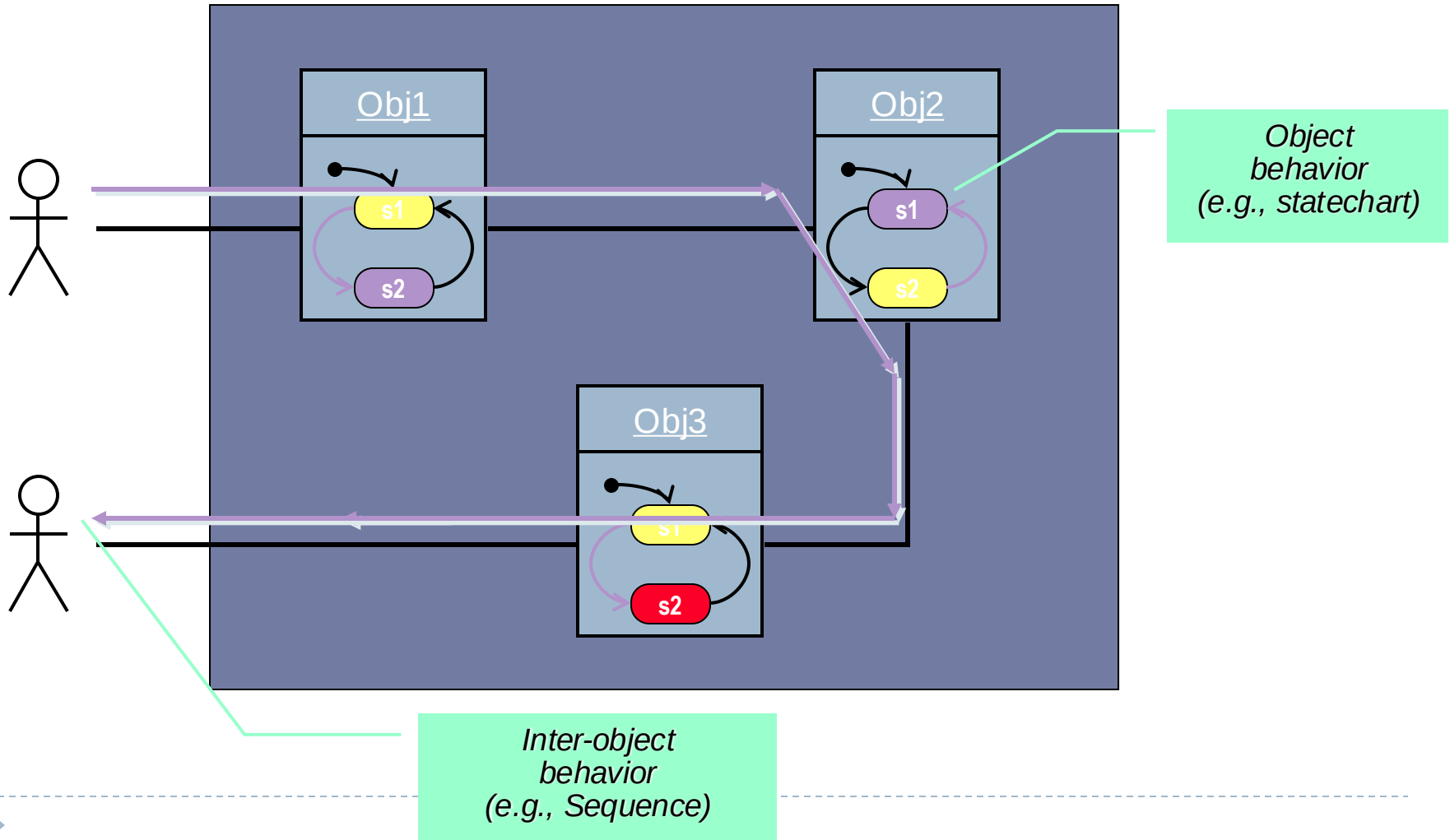


UML Behavioral Models



How behavior can be modeled in UML

- In UML, all behavior results from the actions of (active) objects



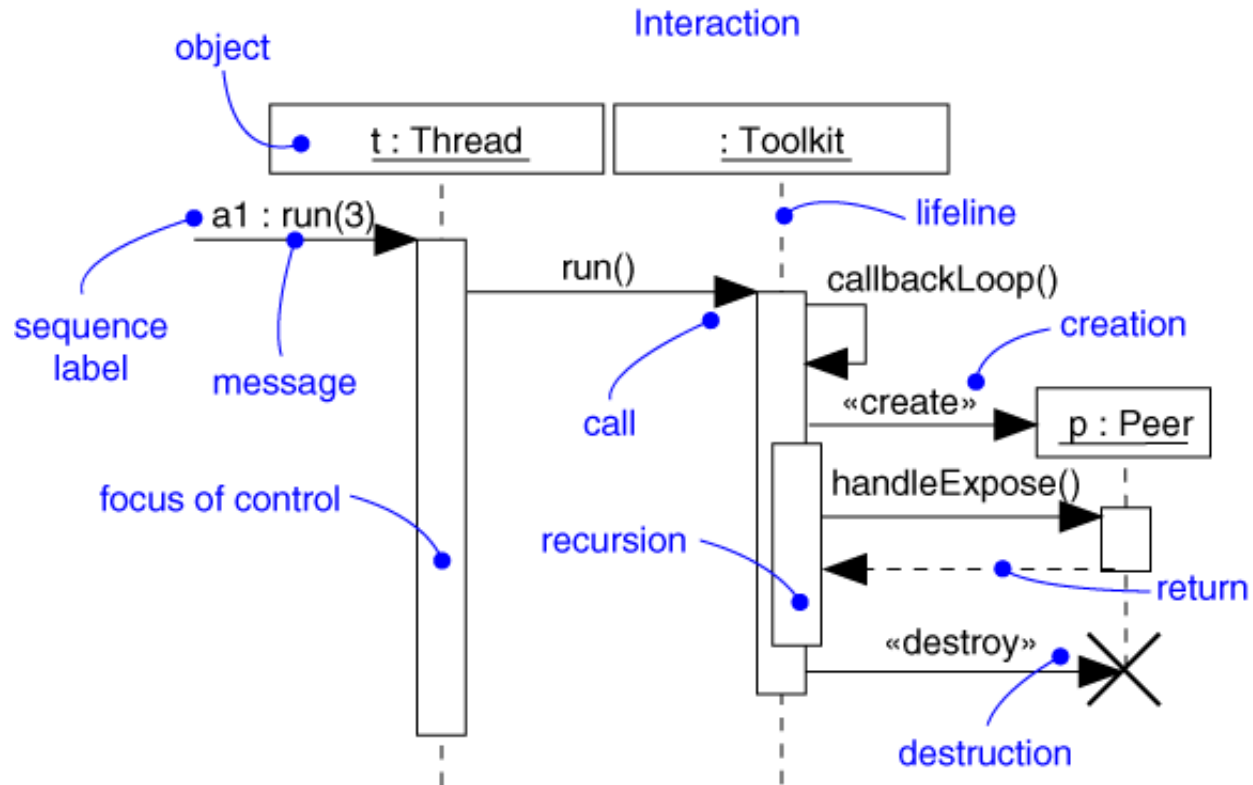
Interaction Diagrams

- ▶ Interaction diagrams are used to model the dynamic aspects of a software system
- ▶ Interaction diagrams show how a set of actors and objects communicate with each other to perform:
 - ▶ The steps of a use case, or
 - ▶ The steps of some other piece of functionality.
- ▶ The set of steps, taken together, is called an *interaction*.
- ▶ Interaction diagrams can show several different types of communication.
 - ▶ E.g. method calls, messages send over the network
 - ▶ These are all referred to as *messages*.

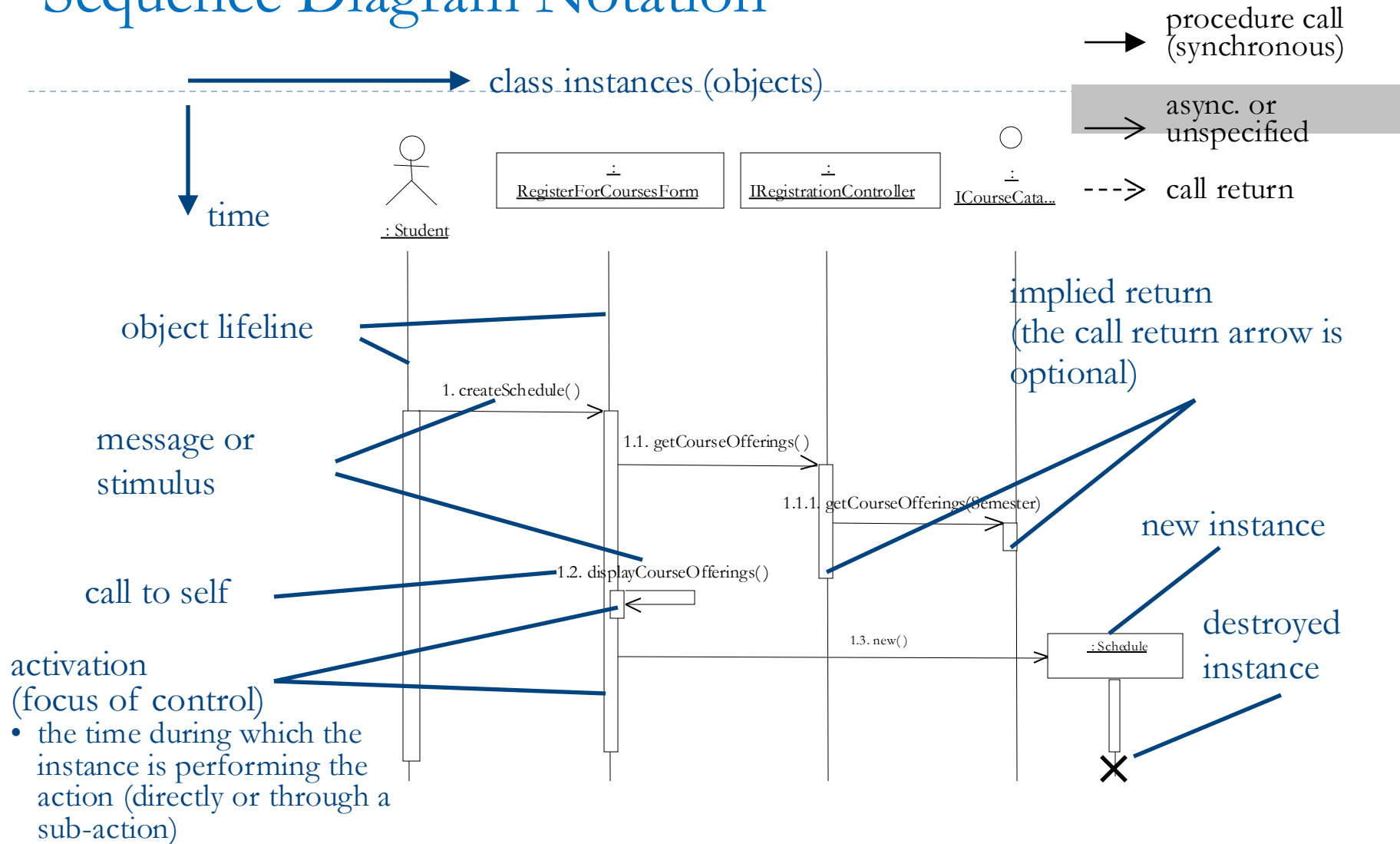


Sequence Diagram

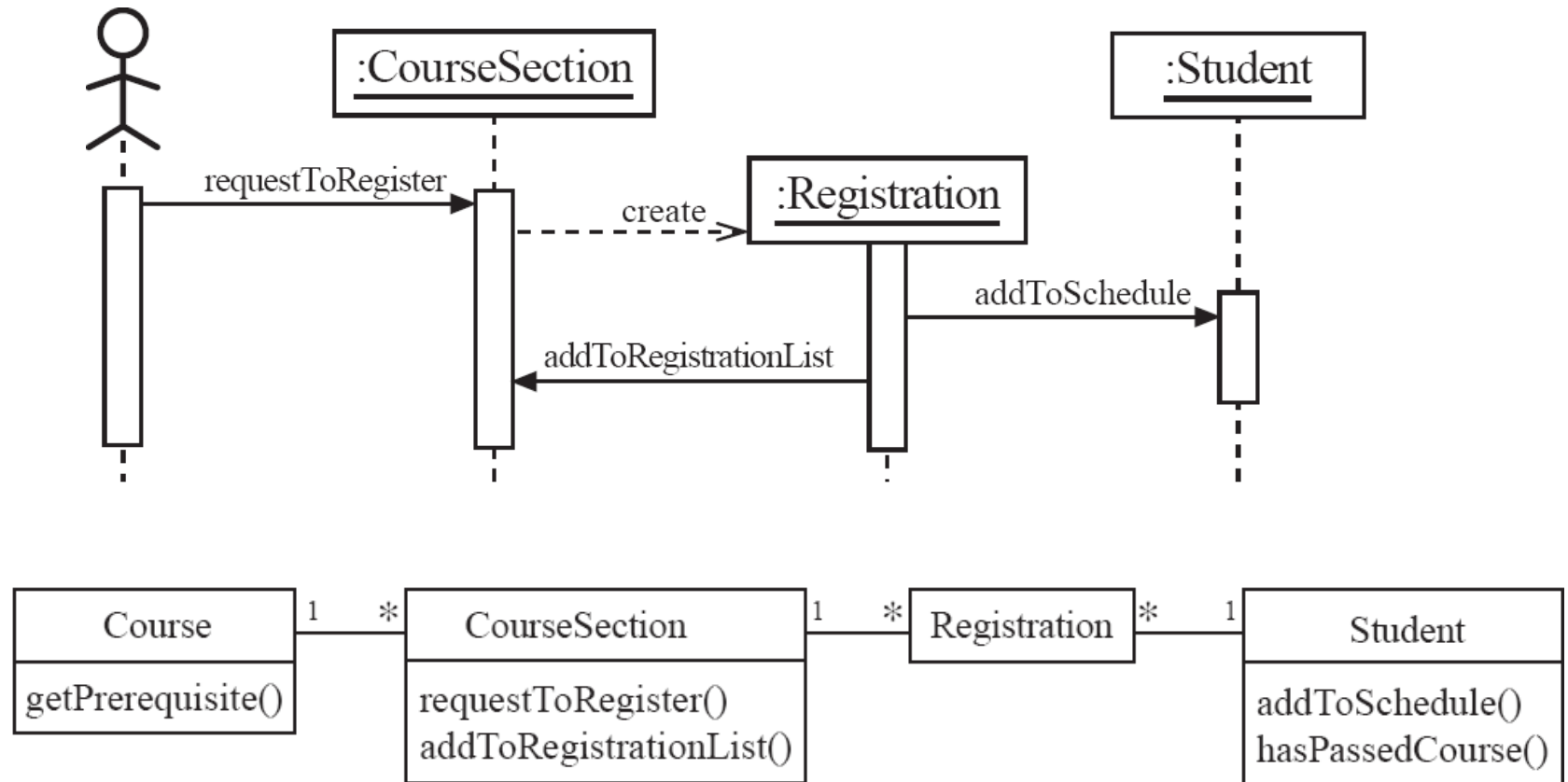
- ▶ Captures dynamic behavior (time-oriented)
- ▶ Purpose
 - ▶ Model flow of control
 - ▶ Illustrate typical scenarios



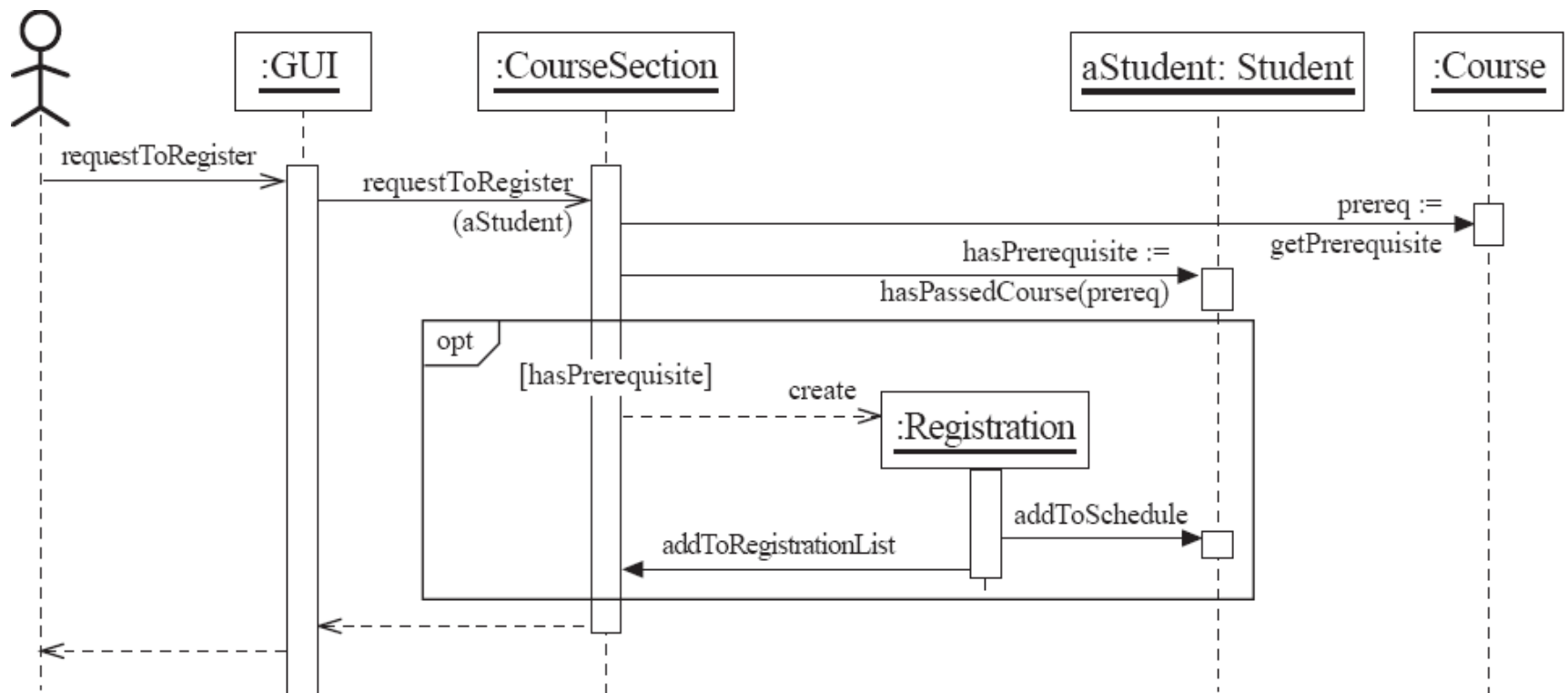
Sequence Diagram Notation



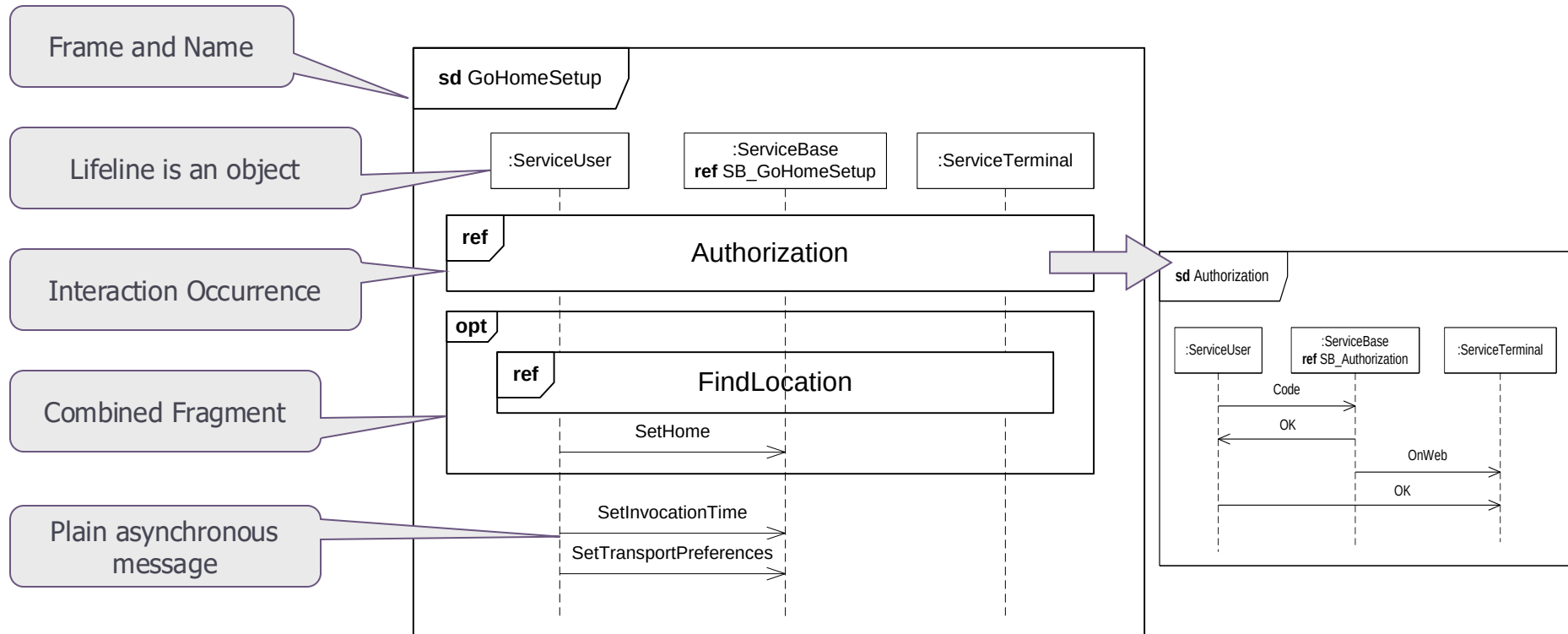
Sequence diagrams – an example



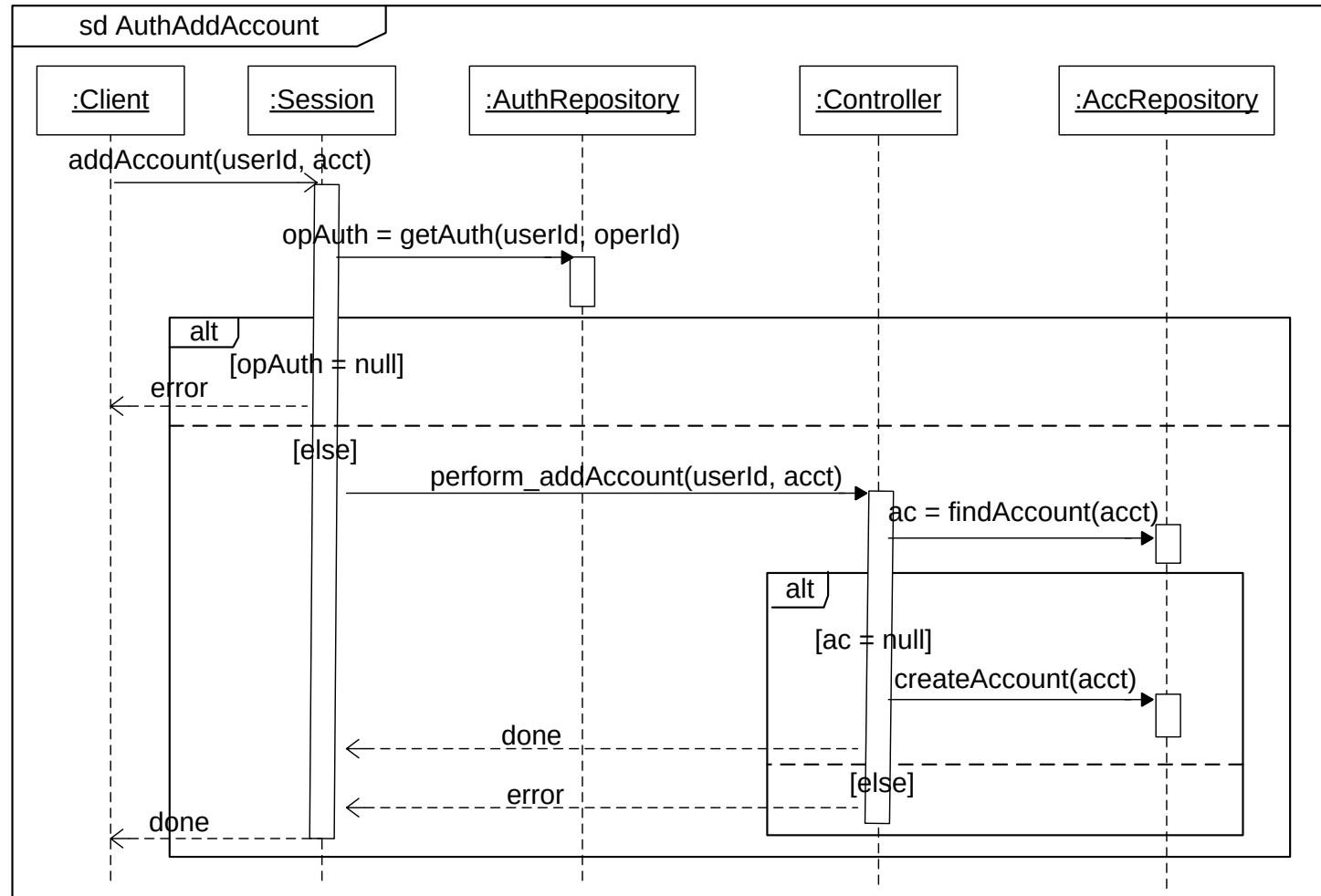
Sequence diagrams – same example, more details



A More Complex Sequence Diagram (UML 2.0)



Sequence diagram - Example



Combined Fragment Types

- ▶ Alternatives (**alt**)
 - ▶ choice of behaviors – at most one will execute
 - ▶ depends on the value of the guard (“else” guard supported)
- ▶ Option (**opt**)
 - ▶ Special case of alternative
- ▶ Break (**break**)
 - ▶ Represents an alternative that is executed instead of the remainder of the fragment (like a break in a loop)
- ▶ Parallel (**par**)
 - ▶ Concurrent (interleaved) sub-scenarios
- ▶ Negative (**neg**)
 - ▶ Identifies sequences that must not occur



Combined Fragment Types

- ▶ Critical Region (**region**)
 - ▶ Traces cannot be interleaved with events on any of the participating lifelines
- ▶ Assertion (**assert**)
 - ▶ Only valid continuation
- ▶ Loop (**loop**)
 - ▶ Optional guard: [<min>, <max>, <Boolean-expression>]
 - ▶ No guard means no specified limit



Modeling using Sequence diagrams -Tips

- ▶ Set the context for the interaction.
- ▶ Express the flow from left to right and from top to bottom.
- ▶ Put active objects to the left/top and passive ones to the right/bottom.
- ▶ Use sequence diagrams
 - ▶ to show the explicit ordering between the stimuli
 - ▶ when modeling real-time





State diagrams



State diagrams

- ▶ A *state diagram* specifies the life histories of objects in terms of the sequences of operations that can occur in response to external stimuli.
 - ▶ For example, a state diagram can describe how an object responds to a request to invoke one of its methods.
- ▶ A state diagram describes behavior in terms of sequences of *states* that an object can go through in response to *events*.



Key Concepts

- ▶ An *event* is a significant or noteworthy occurrence at a point in time.
 - ▶ Examples of events: sending a request to invoke a method, termination of an activity.
 - ▶ An event occurs instantaneously in the time scale of an application.
- ▶ A *state* is a condition of an object during its lifetime.
 - ▶ For example, a student is in the registered state after completing course registration.
 - ▶ A state is an abstraction of an object's attribute values and links
 - ▶ For example, a bank account is in the Overdraft state when the value of its balance attribute is less than 0.

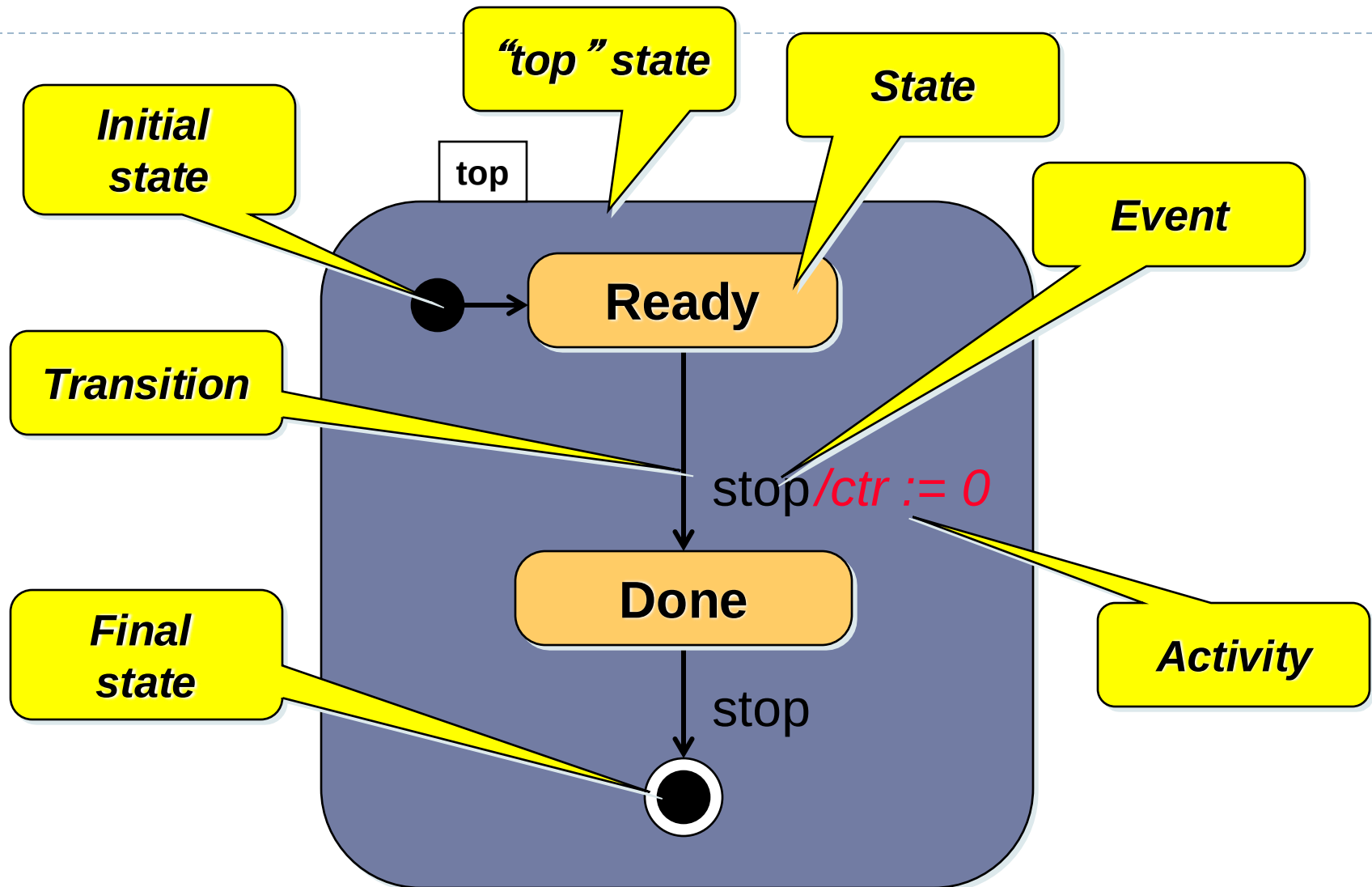


Key Concepts - 2

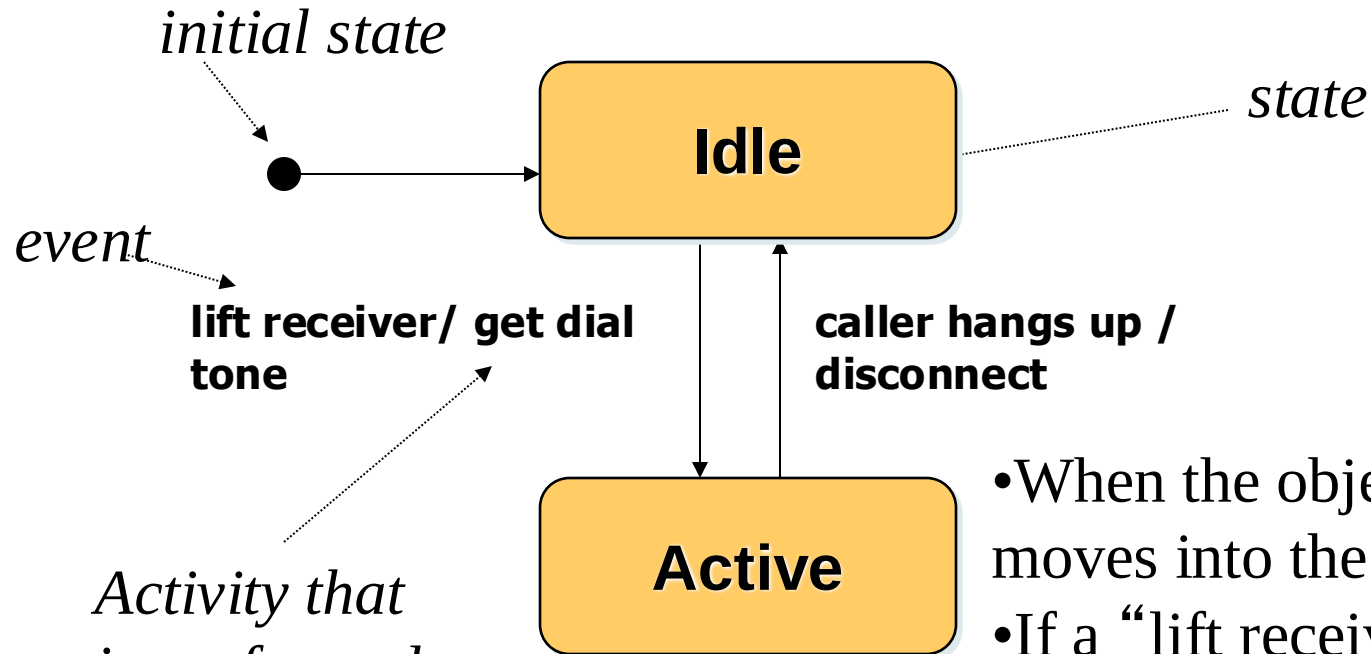
- ▶ A *transition* occurs when an event causes an object to change from its current (source) state to a target state.
 - ▶ For example, if a student is in the registered state and then drops out of the program then the student is in the “not registered” state.
 - ▶ The source and target states can be the same.
 - ▶ A transition is said to fire when the change from source to target state occurs.
 - ▶ A *guard condition* on a transition is a boolean expression that must be true for a transition to fire
 - ▶ An *activity* is a behavior that is executed in response to an event.
-



Basic UML State Diagram



Simple Example: Telephone Object



event

**lift receiver/ /
get dial
tone**

**caller hangs up /
disconnect**

state

Idle

Active

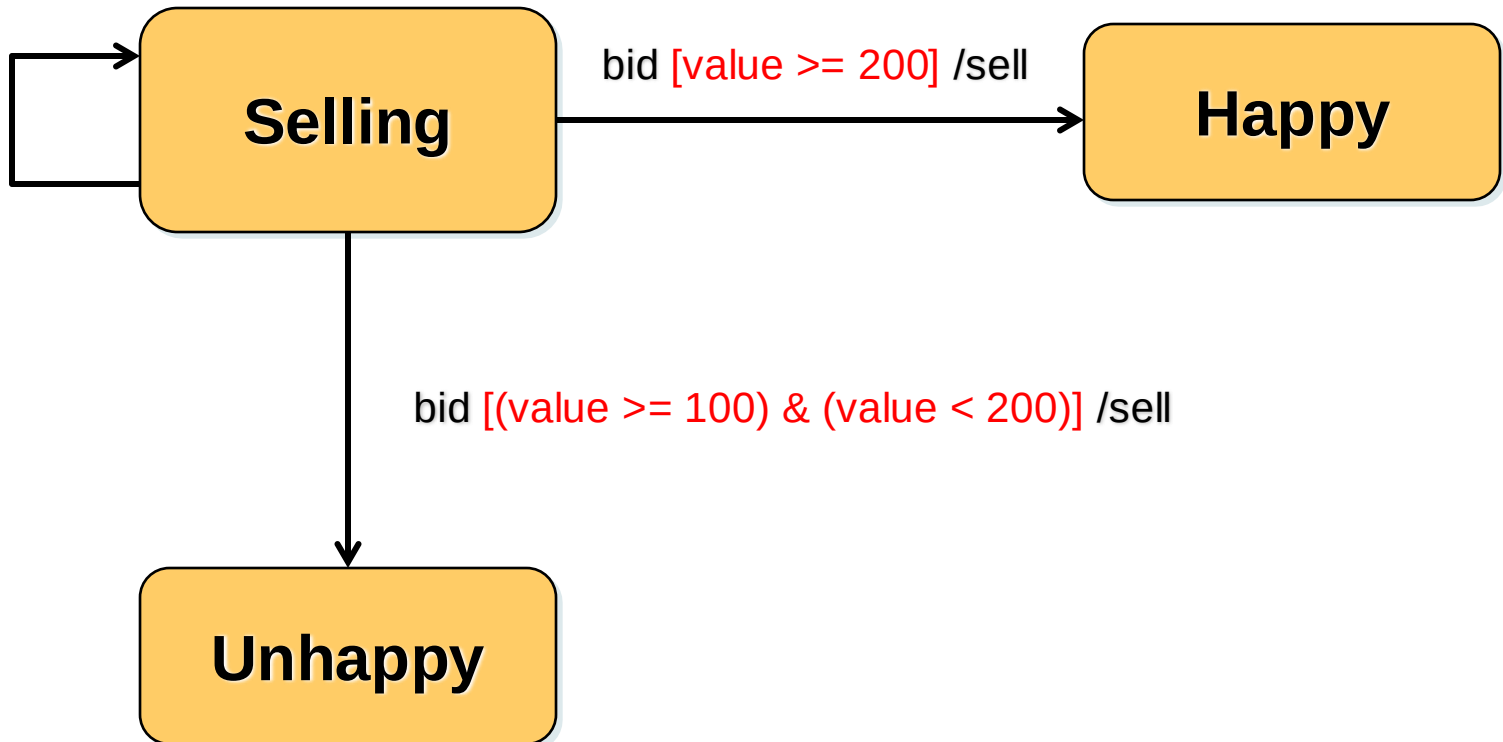
*Activity that
is performed
during
transition: it
executes
instantaneous
ly*

- When the object is created, it moves into the Idle state.
- If a “lift receiver” event is received the object moves from the Idle to the Active state. During the transition the “get dial tone” activity is executed.
- If a “caller hangs up” event occurs when in Active state, the object moves to the Idle state.

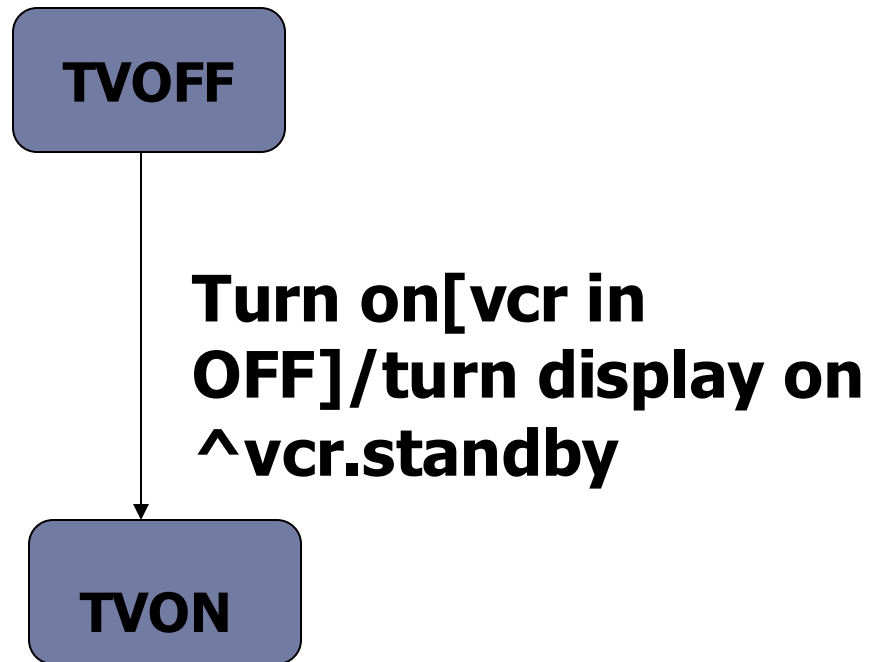
Guards

- ▶ Conditional execution of transitions
 - ▶ guards (Boolean predicates) must be side-effect free

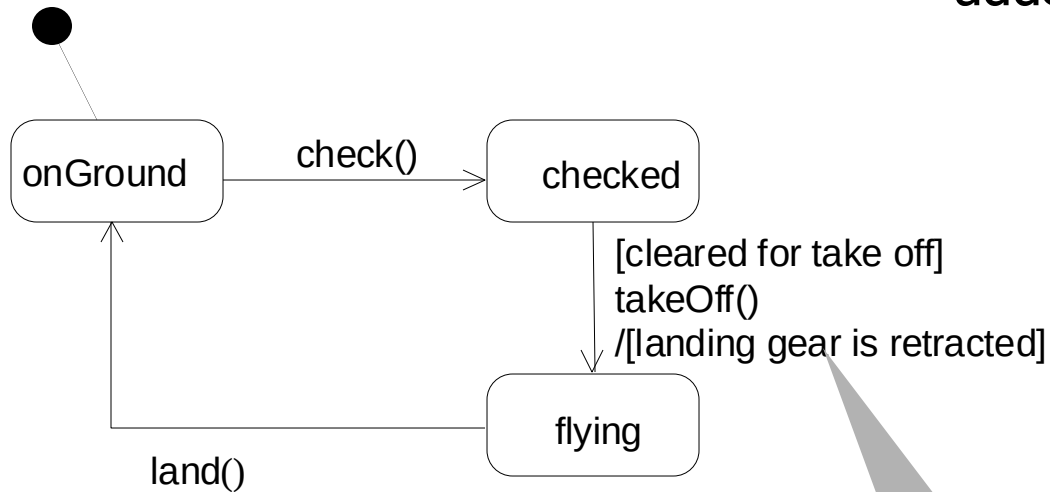
bid [value < 100] /reject



TV - Example



Protocol State Machines



Equivalent to pre and post conditions
added to the related operations:

takeOff()

Pre

- in state "checked"
- cleared for take off

Post

- landing gear is retracted
- in state "flying"

**postcondition
instead of action**

Another Example of a Protocol State Machine

